



UNIVERSITÀ DI PISA

Progetto di Reti Informatiche

Corso di Laurea in Ingegneria Informatica
Anno Accademico 2025/26

Giuseppe Coppola [603355]

Architettura di Sistema

L'applicazione implementata gestisce una lavagna Kanban in modalità ibrida. La **lavagna** vera e propria gestisce la registrazione e l'invio delle card con gli utenti in un sistema **Client-Server**, mentre le operazioni di revisione finali tra **utenti** sono eseguite in **Peer-to-Peer** prima di comunicare l'esito alla lavagna.

Compilazione

Grazie ad un Makefile, per compilare il tutto è sufficiente digitare *make* nella root directory del progetto. Questo andrà a compilare il progetto tramite -Wall per verificare l'assenza di warnings utilizzando i file *utente.c* e *lavagna.c* presenti nella directory *src*, creando i file oggetto .o nella directory *build* e gli eseguibili nella root directory.

Protocollo di Livello Trasporto

Tutte le comunicazioni utilizzano il protocollo **TCP**, a svantaggio di un overhead a causa del 3-way handshake del protocollo rispetto al protocollo UDP. Il TCP è stata l'opzione tecnicamente obbligata in quanto l'intero applicativo è *loss-sensitive*: la perdita occasionale di messaggi (come di un *CARD_DONE* o un *ACK_CARD*) porterebbe irrimediabilmente la lavagna in uno stato inconsistente.

Gestione della Concorrenza (I/O Multiplexing)

Sia lavagna che utente usano **I/O multiplexing** mediante l'uso della funzione *select()*, scelta mirata al fine di utilizzare un singolo flusso di esecuzione. Rispetto ad un approccio multithreading, la select azzerava eventuali race conditions e l'impiego di semafori sulle strutture condivise, conservando l'architettura sicura e più facilmente gestibile.

Tuttavia, un importante svantaggio di questa scelta è la limitata scalabilità intrinseca della *select()*. Questa funzione si basa sulla macro *FD_SETSIZE* che, nei sistemi UNIX classici, limita a 1024 il numero massimo di file descriptor (e quindi di socket) monitorabili contemporaneamente, rendendo il server inadatto ad applicazioni su larghissima scala.

Il listener della lavagna inizializza un socket tipo *SOCK_NONBLOCK* per evitare che rimanga in attesa quando un utente si disconnette poco prima che la lavagna chiami la *accept()*.

Verifica del lavoro (Ping-Pong) e Simulazione del lavoro

Un altro svantaggio dell'utilizzo della *select()* è l'impossibilità dell'utilizzo della funzione *sleep()* per la gestione dei timer di attesa in quanto è una funzione bloccante.

Per verificare che un utente stia lavorando, la lavagna utilizza il timeout della *select* per gestire un contatore dinamico utilizzando il timestamp di ciascuna card.

Nello stesso modo, poiché il processo utente deve rimanere attivo per rispondere ai PING della lavagna durante il lavoro, viene chiamata una funzione *mia_sleep* dopo ogni *select* che gestisce un proprio contatore temporale per simulare il lavoro ma il processo utente non viene mai sospeso.

Protocollo Testuale Vs Binario e File Headers

Le strutture utilizzate implementano il protocollo binario. In modo da minimizzare l'overhead ed evitare la necessità del text parsing. Tuttavia risulta svantaggioso se volessi aggiungere campi alle strutture inviate, in quanto dovrei effettuare più modifiche in varie parti del codice, oppure se volessi debuggare in quanto i dati trasmessi non sono leggibili.

L'architettura utilizza **4 File Header (.h)** principali fortemente modulati:

- **kanban.h:** Contiene costanti globali del progetto e la definizione della struttura dati centrale Card.
- **commands.h:** Contiene i *define* per tutti i comandi che è possibile inviare/utilizzare.
- **protocol.h:** Definisce delle costanti di rete e le strutture inviate. Utilizzo il costrutto `__attribute__((packed))` sulle struct inviate forzare il compilatore ad eliminare il padding per rendere precisa la grandezza inviata via socket. Costrutto utilizzato da GCC, non funzionerebbe in sistemi Windows. Ogni messaggio possiede un Header formato da <Comando, Lunghezza del Payload> a precedere i dati.
- **utente.h / lavagna.h:** Costanti, strutture e definizione di funzioni utilizzate rispettivamente solo dall'utente/lavagna.

Uso del terminale

Oltre alle operazioni automatiche, il sistema supporta anche alcuni comandi utilizzabili da terminale.

La **Lavagna** può utilizzare:

- **SHOW_LAVAGNA:** Mostra sul terminale lo stato attuale della lavagna.
- **HANDLE_CARD:** Spedisce pro-attivamente le notifiche automatiche di invio Card verso utenti che denotano stasi (stato "TODO" → "DOING"). Utile solo per eventuale debug, in quanto la gestione delle card è automatico.

L'**Utente** può utilizzare:

- **HELLO:** Per registrare la propria presenza alla lavagna.
- **CREATE_CARD:** Permette di creare una Card da inviare alla lavagna, la quale la aggiungerà alla colonna TODO.
- **ACK_CARD:** Serve per confermare alla lavagna la corretta ricezione della card.
- **QUIT:** Serve per comunicare alla lavagna la propria uscita.
- **REQUEST_USER_LIST:** Chiede alla lavagna una lista degli utenti connessi.
- **REVIEW_CARD:** Utilizzabile solo se la prima review automatica fallisce, permette di forzare un tentativo di review prima del tempo di attesa predefinito.

Debug

Per il debug e la verifica di ogni operazione effettuata dalla lavagna/utente è possibile settare a 1 la variabile `DEBUG_LOG` presente nel file *kanban.h* per mostrare il log di tutte le operazioni.